

LAPORAN PRAKTIKUM SISTEM OPERASI

Dosen Pengampu :

TRIAWAN ADI CAHYANTO M.Kom



Disusun Oleh:

Pradana Bagus Susilo (2410651007)

PROGRAM STUDI TEKNIK INFORMATIKA

FAKULTAS TEKNIK

UNIVERSITAS MUHAMMADIYAH JEMBER

2025

Tujuan

1. Memahami Konsep Dasar Sistem Operasi

- Menjelaskan fungsi utama sistem operasi, seperti manajemen proses, memori, dan perangkat keras.
- Membedakan antara kernel, shell, dan distribusi Linux (misalnya, Ubuntu, CentOS).
- Mengidentifikasi komponen utama Linux, termasuk file system (ext4, Btrfs) dan boot process.

2. Menguasai Command Line Interface (CLI)

- Menggunakan perintah dasar seperti ls, cd, mkdir, cp, mv, dan rm untuk navigasi dan manipulasi file.
- Memahami penggunaan pipes, redirection, dan wildcards untuk efisiensi kerja.
- Menjalankan perintah dengan opsi lanjutan, seperti grep, find, dan sed untuk pencarian dan pengeditan teks.

3. Mengelola File dan Direktori

- Mengatur izin file dan direktori menggunakan chmod, chown, dan chgrp.
- Memahami struktur file system Linux (/root, /home, /etc, dll.) dan mounting perangkat.
- Menggunakan alat seperti tar, gzip, dan rsync untuk kompresi dan sinkronisasi file.

4. Menginstal dan Mengkonfigurasi Software

- Menggunakan package manager seperti apt (Debian-based) atau yum/dnf (Red Hat-based) untuk instalasi aplikasi.
- Mengkompilasi software dari source code menggunakan make dan configure.
- Mengelola dependensi dan repositori perangkat lunak.

5. Mengelola Proses dan Pengguna

- Memantau dan mengontrol proses dengan ps, top, kill, dan nice.
- Membuat dan mengelola akun pengguna serta grup menggunakan useradd, usermod, dan passwd.
- Mengonfigurasi layanan sistem seperti systemd untuk startup otomatis.

6. Dasar-Dasar Jaringan dan Keamanan

- Mengkonfigurasi jaringan dengan ifconfig atau ip, termasuk pengaturan IP, DNS, dan firewall (iptables atau firewallld).
- Memahami protokol dasar seperti SSH untuk akses jarak jauh dan SCP untuk transfer file.

Tugas 1:Eksperimen dengan data berbeda

- Source Kode:

```
#include <stdio.h>
```

```
struct Process {  
    int pid;  
    int at;  
    int bt;  
    int wt;  
    int tat;  
    int rt; // Remaining time for RR  
};
```

```
// ----- FCFS -----
```

```
void FCFS(struct Process p[], int n) {  
    int time = 0;  
    float total_wt = 0, total_tat = 0;  
  
    printf("\n=== FCFS ===\n");  
    for (int i = 0; i < n; i++) {  
        if (time < p[i].at)  
            time = p[i].at;  
        p[i].wt = time - p[i].at;  
        time += p[i].bt;  
        p[i].tat = p[i].wt + p[i].bt;  
        total_wt += p[i].wt;  
        total_tat += p[i].tat;  
    }  
  
    printf("PID\tAT\tBT\tWT\tTAT\n");  
    for (int i = 0; i < n; i++)  
        printf("P%d\t%d\t%d\t%d\t%d\n", p[i].pid, p[i].at, p[i].bt, p[i].wt, p[i].tat);  
  
    printf("Average Waiting Time = %.2f\n", total_wt / n);  
    printf("Average Turnaround Time = %.2f\n", total_tat / n);  
}
```

```
// ----- SJF (Non-preemptive) -----
```

```
void SJF(struct Process p[], int n) {  
    int completed = 0, time = 0;  
    float total_wt = 0, total_tat = 0;  
    int done[10] = {0};  
  
    printf("\n=== SJF (Non-preemptive) ===\n");
```

```

while (completed < n) {
    int idx = -1, min_bt = 9999;
    for (int i = 0; i < n; i++) {
        if (!done[i] && p[i].at <= time && p[i].bt < min_bt) {
            min_bt = p[i].bt;
            idx = i;
        }
    }

    if (idx == -1) {
        time++;
        continue;
    }

    p[idx].wt = time - p[idx].at;
    time += p[idx].bt;
    p[idx].tat = p[idx].wt + p[idx].bt;
    done[idx] = 1;
    completed++;

    total_wt += p[idx].wt;
    total_tat += p[idx].tat;
}

printf("PID\tAT\tBT\tWT\tTAT\n");
for (int i = 0; i < n; i++)
    printf("P%d\t%d\t%d\t%d\t%d\n", p[i].pid, p[i].at, p[i].bt, p[i].wt, p[i].tat);

printf("Average Waiting Time = %.2f\n", total_wt / n);
printf("Average Turnaround Time = %.2f\n", total_tat / n);
}

// ----- Round Robin -----
void RoundRobin(struct Process p[], int n, int quantum) {
    int rem_bt[10];
    for (int i = 0; i < n; i++) rem_bt[i] = p[i].bt;

    int time = 0, completed = 0;
    float total_wt = 0, total_tat = 0;

    printf("\n=== Round Robin (q=%d) ===\n", quantum);
    while (completed < n) {
        int done = 1;
        for (int i = 0; i < n; i++) {
            if (rem_bt[i] > 0 && p[i].at <= time) {

```

```

        done = 0;
        if (rem_bt[i] > quantum) {
            time += quantum;
            rem_bt[i] -= quantum;
        } else {
            time += rem_bt[i];
            p[i].wt = time - p[i].bt - p[i].at;
            p[i].tat = p[i].wt + p[i].bt;
            rem_bt[i] = 0;
            completed++;
            total_wt += p[i].wt;
            total_tat += p[i].tat;
        }
    }
}
if (done) time++;
}

printf("PID\tAT\tBT\tWT\tTAT\n");
for (int i = 0; i < n; i++)
    printf("P%d\t%d\t%d\t%d\t%d\n", p[i].pid, p[i].at, p[i].bt, p[i].wt, p[i].tat);

printf("Average Waiting Time = %.2f\n", total_wt / n);
printf("Average Turnaround Time = %.2f\n", total_tat / n);
}

// ----- Main -----
int main() {
    struct Process p1[] = {
        {1, 0, 10, 0, 0, 0},
        {2, 1, 5, 0, 0, 0},
        {3, 2, 8, 0, 0, 0}
    };
    int n1 = 3;

    struct Process p2[] = {
        {1, 0, 5, 0, 0, 0},
        {2, 1, 3, 0, 0, 0},
        {3, 2, 8, 0, 0, 0},
        {4, 3, 6, 0, 0, 0}
    };
    int n2 = 4;

    printf("\n===== DATA SET 1 =====");
    FCFS(p1, n1);

```

```

SJF(p1, n1);
RoundRobin(p1, n1, 3);

printf("\n\n===== DATA SET 2 =====");
FCFS(p2, n2);
SJF(p2, n2);
RoundRobin(p2, n2, 3);

return 0;
}

```

- Output

Data 1

```

pradana@Bagus:~$ nano cpu_scheduling.c
pradana@Bagus:~$ gcc cpu_scheduling.c -o cpu
./cpu

===== DATA SET 1 =====
=== FCFS ===
PID    AT     BT     WT     TAT
P1      0      10     0      10
P2      1       5     9      14
P3      2       8    13      21
Average Waiting Time = 7.33
Average Turnaround Time = 15.00

=== SJF (Non-preemptive) ===
PID    AT     BT     WT     TAT
P1      0      10     0      10
P2      1       5     9      14
P3      2       8    13      21
Average Waiting Time = 7.33
Average Turnaround Time = 15.00

=== Round Robin (q=3) ===
PID    AT     BT     WT     TAT
P1      0      10    13      23
P2      1       5     8      13
P3      2       8    12      20
Average Waiting Time = 11.00
Average Turnaround Time = 18.67

```

- Data 2

```

===== DATA SET 2 =====
=== FCFS ===
PID    AT     BT     WT     TAT
P1      0       5     0       5
P2      1       3     4       7
P3      2       8     6      14
P4      3       6    13      19
Average Waiting Time = 5.75
Average Turnaround Time = 11.25

=== SJF (Non-preemptive) ===
PID    AT     BT     WT     TAT
P1      0       5     0       5
P2      1       3     4       7
P3      2       8    12      20
P4      3       6     5      11
Average Waiting Time = 5.25
Average Turnaround Time = 10.75

=== Round Robin (q=3) ===
PID    AT     BT     WT     TAT
P1      0       5     9      14
P2      1       3     2       5
P3      2       8    12      20
P4      3       6    11      17
Average Waiting Time = 8.50
Average Turnaround Time = 14.00
pradana@Bagus:~$ |

```

Tugas 2 : Analisis Eksperimen ketiga algoritma

1. Algoritma mana yang memberikan Average Waiting Time terkecil?

Jawab:

SJF (Shortest Job First) memberikan Average Waiting Time paling kecil, karena dia memprioritaskan proses dengan waktu eksekusi (burst) paling pendek dulu.

2. Mengapa hasil algoritma berbeda-beda?

Jawab:

Karena cara memilih proses berikutnya berbeda-beda

- a. FCFS yang datang lebih dulu dijalankan duluan, walau butuh waktu lama.
- b. SJF yang waktu prosesnya paling pendek dijalankan duluan.
- c. Round Robin tiap proses dapat giliran bergantian (time slice), cocok untuk sistem multitasking

3. Kapan sebaiknya menggunakan algoritma FCFS?

Gunakan FCFS jika:

- a. Semua proses punya waktu hampir sama (tidak ada yang jauh lebih panjang).
- b. Sistem sederhana dan tidak butuh respon cepat (misalnya sistem batch)

4. Kapan sebaiknya menggunakan algoritma SJF?

Gunakan SJF jika:

- a. Kamu tahu perkiraan waktu eksekusi setiap proses.
- b. Tujuanmu adalah meminimalkan waktu tunggu total.
- c. Sistem tidak memerlukan interaksi real-time.

5. Kapan sebaiknya menggunakan algoritma Round Robin?

Gunakan Round Robin jika:

- a. Sistem kamu multitasking / time-sharing, di mana semua proses harus mendapat giliran.
- b. Kamu ingin respon cepat untuk banyak user.
- c. Cocok untuk sistem interaktif seperti OS modern (Ubuntu, Windows, dll).

Tugas 3 : Modifikasi Program

Disini saya akan memodifikasi pada program “compare.c” dengan fitur fitur berikut.

1. Tampilkan Gantt Chart yang lebih detail
2. Simpan hasil ke file text
3. Baca input dari file

- Source Kode

```
#include <stdio.h>
#include <stdbool.h>
#include <string.h>
```

```
#define MAX 100
```

```
// Struktur data proses
```

```
struct Process {
    int pid;
    int arrival_time;
    int burst_time;
    int waiting_time;
    int turnaround_time;
    int completion_time;
    int remaining_time;
    bool is_completed;
};
```

```
// Struktur hasil algoritma
```

```
struct AlgorithmResult {
    char name[40];
    float avg_waiting_time;
    float avg_turnaround_time;
    int total_context_switches;
};
```

```
// Fungsi reset proses
```

```
void reset_processes(struct Process p[], struct Process backup[], int n) {
    for (int i = 0; i < n; i++) {
        p[i] = backup[i];
        p[i].waiting_time = 0;
        p[i].turnaround_time = 0;
        p[i].completion_time = 0;
        p[i].remaining_time = p[i].burst_time;
        p[i].is_completed = false;
    }
}
```



```

// =====
// Fungsi menampilkan tabel hasil tiap algoritma
// =====
void display_results(struct Process p[], int n, char *algo) {
    float total_tat = 0, total_wt = 0;

    printf("\n=====
    ==\n");
    printf(" HASIL ALGORITMA %s\n", algo);

    printf("=====
    \n");
    printf("PID\tAT\tBT\tCT\tTAT\tWT\n");
    printf("-----\n");
    for (int i = 0; i < n; i++) {
        printf("%d\t%d\t%d\t%d\t%d\t%d\n",
            p[i].pid, p[i].arrival_time, p[i].burst_time,
            p[i].completion_time, p[i].turnaround_time, p[i].waiting_time);
        total_tat += p[i].turnaround_time;
        total_wt += p[i].waiting_time;
    }
    printf("-----\n");
    printf("Rata-rata Turnaround Time: %.2f\n", total_tat / n);
    printf("Rata-rata Waiting Time : %.2f\n", total_wt / n);
}

// =====
// Gantt Chart sederhana
// =====
void display_gantt_chart(struct Process p[], int n, char *algo) {
    printf("\nGantt Chart (%s):\n", algo);
    printf(" ");
    for (int i = 0; i < n; i++) printf("-----");
    printf("\n|");
    for (int i = 0; i < n; i++) printf(" P%d |", p[i].pid);
    printf("\n ");
    for (int i = 0; i < n; i++) printf("-----");
    printf("\n0");
    for (int i = 0; i < n; i++) printf("\t%d", p[i].completion_time);
    printf("\n");
}

// =====
// FCFS
// =====

```

```

struct AlgorithmResult fcfs(struct Process p[], int n) {
    struct AlgorithmResult r;
    strcpy(r.name, "FCFS");
    int time = 0;
    float total_tat = 0, total_wt = 0;

    for (int i = 0; i < n; i++) {
        if (time < p[i].arrival_time) time = p[i].arrival_time;
        p[i].completion_time = time + p[i].burst_time;
        p[i].turnaround_time = p[i].completion_time - p[i].arrival_time;
        p[i].waiting_time = p[i].turnaround_time - p[i].burst_time;
        time = p[i].completion_time;
        total_tat += p[i].turnaround_time;
        total_wt += p[i].waiting_time;
    }

    r.avg_turnaround_time = total_tat / n;
    r.avg_waiting_time = total_wt / n;
    r.total_context_switches = n - 1;

    display_results(p, n, "FCFS");
    display_gantt_chart(p, n, "FCFS");
    return r;
}

// =====
// SJF Non-preemptive
// =====

struct AlgorithmResult sjf(struct Process p[], int n) {
    struct AlgorithmResult r;
    strcpy(r.name, "SJF");
    int completed = 0, time = 0, cs = 0;
    float total_tat = 0, total_wt = 0;

    while (completed < n) {
        int idx = -1, min_burst = 1e9;
        for (int i = 0; i < n; i++) {
            if (!p[i].is_completed && p[i].arrival_time <= time && p[i].burst_time <
min_burst) {
                min_burst = p[i].burst_time;
                idx = i;
            }
        }

        if (idx == -1) {

```

```

        time++;
        continue;
    }

    time += p[idx].burst_time;
    p[idx].completion_time = time;
    p[idx].turnaround_time = p[idx].completion_time - p[idx].arrival_time;
    p[idx].waiting_time = p[idx].turnaround_time - p[idx].burst_time;
    p[idx].is_completed = true;
    completed++;
    total_tat += p[idx].turnaround_time;
    total_wt += p[idx].waiting_time;
    if (completed < n) cs++;
}

r.avg_turnaround_time = total_tat / n;
r.avg_waiting_time = total_wt / n;
r.total_context_switches = cs;

display_results(p, n, "SJF");
display_gantt_chart(p, n, "SJF");
return r;
}

// =====
// Round Robin
// =====

struct AlgorithmResult rr(struct Process p[], int n, int q) {
    struct AlgorithmResult r;
    sprintf(r.name, "Round Robin (Q=%d)", q);
    int time = 0, completed = 0, cs = 0;
    float total_tat = 0, total_wt = 0;
    int queue[MAX], front = 0, rear = 0;
    bool in_q[MAX] = {false};

    while (completed < n) {
        for (int i = 0; i < n; i++) {
            if (!in_q[i] && !p[i].is_completed && p[i].arrival_time <= time) {
                queue[rear++] = i;
                in_q[i] = true;
            }
        }
    }

    if (front == rear) {
        time++;
    }
}

```

```

        continue;
    }

    int idx = queue[front++];
    in_q[idx] = false;

    if (p[idx].remaining_time <= q) {
        time += p[idx].remaining_time;
        p[idx].remaining_time = 0;
        p[idx].completion_time = time;
        p[idx].turnaround_time = p[idx].completion_time - p[idx].arrival_time;
        p[idx].waiting_time = p[idx].turnaround_time - p[idx].burst_time;
        p[idx].is_completed = true;
        total_tat += p[idx].turnaround_time;
        total_wt += p[idx].waiting_time;
        completed++;
    } else {
        time += q;
        p[idx].remaining_time -= q;
        cs++;
    }
}

r.avg_turnaround_time = total_tat / n;
r.avg_waiting_time = total_wt / n;
r.total_context_switches = cs;

display_results(p, n, r.name);
display_gantt_chart(p, n, r.name);
return r;
}

// =====
// MAIN PROGRAM
// =====

int main() {
    int n, quantum;
    struct Process p[MAX], backup[MAX];
    FILE *finput = fopen("input.txt", "r");

    printf("=====
\n");
    printf("  PROGRAM PERBANDINGAN ALGORITMA CPU SCHEDULING\n");

```

```

printf("=====
\n");

if (finput) {
    printf("Membaca data dari file input.txt ...\n");
    fscanf(finput, "%d %d", &n, &quantum);
    for (int i = 0; i < n; i++) {
        fscanf(finput, "%d %d", &p[i].arrival_time, &p[i].burst_time);
        p[i].pid = i + 1;
    }
    fclose(finput);
} else {
    printf("Masukkan jumlah proses: ");
    scanf("%d", &n);
    printf("Masukkan time quantum (RR): ");
    scanf("%d", &quantum);
    for (int i = 0; i < n; i++) {
        p[i].pid = i + 1;
        printf("\nProses %d\n", i + 1);
        printf("Arrival Time: ");
        scanf("%d", &p[i].arrival_time);
        printf("Burst Time: ");
        scanf("%d", &p[i].burst_time);
    }
}

for (int i = 0; i < n; i++) backup[i] = p[i];

printf("\n=====
==\n");

reset_processes(p, backup, n);
struct AlgorithmResult fc = fcfs(p, n);

reset_processes(p, backup, n);
struct AlgorithmResult sj = sjf(p, n);

reset_processes(p, backup, n);
struct AlgorithmResult rrres = rr(p, n, quantum);

// =====
// SIMPAN HASIL KE FILE TEKS
// =====

```

```

FILE *f = fopen("hasil_compare.txt", "w");
fprintf(f,
"=====\\n");
fprintf(f, "      HASIL PERBANDINGAN ALGORITMA CPU SCHEDULING\\n");
fprintf(f,
"=====\\n\\n");
fprintf(f, "%-25s | %-10s | %-10s | %-15s\\n",
      "Algoritma", "Avg TAT", "Avg WT", "Context Switch");
fprintf(f, "-----\\n");
fprintf(f, "%-25s | %-10.2f | %-10.2f | %-15d\\n",
      fc.name, fc.avg_turnaround_time, fc.avg_waiting_time, fc.total_context_switches);
fprintf(f, "%-25s | %-10.2f | %-10.2f | %-15d\\n",
      sj.name, sj.avg_turnaround_time, sj.avg_waiting_time, sj.total_context_switches);
fprintf(f, "%-25s | %-10.2f | %-10.2f | %-15d\\n",
      rrres.name, rrres.avg_turnaround_time, rrres.avg_waiting_time,
rrres.total_context_switches);
fprintf(f,
"=====\\n");
fclose(f);

printf("\\nHasil perbandingan disimpan ke file 'hasil_compare.txt'\\n");

printf("=====\\n");

return 0;
}

```

- Output:
Jika dijalankan dengan file txt data set 1 maka hasilnya

```
pradana@Bagus:~$ nano compare.c
pradana@Bagus:~$ nano input1.txt
pradana@Bagus:~$ nano input2.txt
pradana@Bagus:~$ cp input1.txt input.txt
./compare
=====
PROGRAM PERBANDINGAN ALGORITMA CPU SCHEDULING
=====
Membaca data dari file input.txt ...
=====
HASIL ALGORITMA FCFS
=====
PID  AT   BT   CT   TAT  WT
-----
1    0    10   10   10   0
2    1    5    15   14   9
3    2    8    23   21   13
=====
Rata-rata Turnaround Time: 15.00
Rata-rata Waiting Time : 7.33
Gantt Chart (FCFS):
-----
| P1 | P2 | P3 |
-----
0    10   15   23
=====
HASIL ALGORITMA SJF
=====
PID  AT   BT   CT   TAT  WT
-----
1    0    10   10   10   0
2    1    5    15   14   9
3    2    8    23   21   13
=====
Rata-rata Turnaround Time: 15.00
Rata-rata Waiting Time : 7.33
```

```
Gantt Chart (SJF):
-----
| P1 | P2 | P3 |
-----
0    10   15   23
=====
HASIL ALGORITMA Round Robin (Q=2)
=====
PID  AT   BT   CT   TAT  WT
-----
1    0    10   21   21   11
2    1    5    17   16   11
3    2    8    23   21   13
=====
Rata-rata Turnaround Time: 19.33
Rata-rata Waiting Time : 11.67
Gantt Chart (Round Robin (Q=2)):
-----
| P1 | P2 | P3 |
-----
0    21   17   23
=====
Hasil perbandingan disimpan ke file 'hasil_compare.txt'
=====
pradana@Bagus:~$ |
```

Jika dijalankan dengan file txt data set 2 maka hasilnya:

```
pradana@Bagus:~$ cp input2.txt input.txt
./compare
=====
PROGRAM PERBANDINGAN ALGORITMA CPU SCHEDULING
=====
Membaca data dari file input.txt ...
=====
HASIL ALGORITMA FCFS
=====
PID  AT   BT   CT   TAT  WT
-----
1    0    5    5    5    0
2    1    8    13   12   4
3    2    3    16   14   11
4    3    6    22   19   13
=====
Rata-rata Turnaround Time: 12.50
Rata-rata Waiting Time : 7.00
Gantt Chart (FCFS):
-----
| P1 | P2 | P3 | P4 |
-----
0    5    13   16   22
=====
HASIL ALGORITMA SJF
=====
PID  AT   BT   CT   TAT  WT
-----
1    0    5    5    5    0
2    1    8    22   21   13
3    2    3    8    6    3
4    3    6    14   11   5
=====
Rata-rata Turnaround Time: 10.75
Rata-rata Waiting Time : 5.25
```

```
Gantt Chart (SJF):
-----
| P1 | P2 | P3 | P4 |
-----
0    5    22   8    14
=====
HASIL ALGORITMA Round Robin (Q=2)
=====
PID  AT   BT   CT   TAT  WT
-----
1    0    5    9    9    4
2    1    8    22   21   13
3    2    3    14   12   9
4    3    6    20   17   11
=====
Rata-rata Turnaround Time: 14.75
Rata-rata Waiting Time : 9.25
Gantt Chart (Round Robin (Q=2)):
-----
| P1 | P2 | P3 | P4 |
-----
0    9    22   14   20
=====
Hasil perbandingan disimpan ke file 'hasil_compare.txt'
=====
pradana@Bagus:~$ |
```

Tugas 4: Eksperimen dengan data berbeda untuk perbandingan algoritma

- Test case 1:

```
pradana@Bagus:~$ nano input1.txt
pradana@Bagus:~$ cp input1.txt input.txt
./compare
=====
PROGRAM PERBANDINGAN ALGORITMA CPU SCHEDULING
=====
Membaca data dari file input.txt ...
=====
HASIL ALGORITMA FCFS
=====
PID  AT   BT   CT   TAT  WT
-----
1    0   24   24   24   0
2    1    3   27   26   23
3    2    3   30   28   25
4    3    6   36   33   27
=====
Rata-rata Turnaround Time: 27.75
Rata-rata Waiting Time : 18.75
Gantt Chart (FCFS):
| P1 | P2 | P3 | P4 |
0    24   27   30   36
=====
HASIL ALGORITMA SJF
=====
PID  AT   BT   CT   TAT  WT
-----
1    0   24   24   24   0
2    1    3   27   26   23
3    2    3   30   28   25
4    3    6   36   33   27
=====
Rata-rata Turnaround Time: 27.75
Rata-rata Waiting Time : 18.75
```

```
Gantt Chart (SJF):
| P1 | P2 | P3 | P4 |
0    24   27   30   36
=====
HASIL ALGORITMA Round Robin (Q=3)
=====
PID  AT   BT   CT   TAT  WT
-----
1    0   24   36   36   12
2    1    3    9    8    5
3    2    3   12   10    7
4    3    6   21   18   12
=====
Rata-rata Turnaround Time: 18.00
Rata-rata Waiting Time : 9.00
Gantt Chart (Round Robin (Q=3)):
| P1 | P2 | P3 | P4 |
0    36    9   12   21
=====
Hasil perbandingan disimpan ke file 'hasil_compare.txt'
=====
pradana@Bagus:~$ |
```

- Test Case 2:

```
pradana@Bagus:~$ nano input2.txt
pradana@Bagus:~$ cp input2.txt input.txt
./compare
=====
PROGRAM PERBANDINGAN ALGORITMA CPU SCHEDULING
=====
Membaca data dari file input.txt ...
=====
HASIL ALGORITMA FCFS
=====
PID  AT   BT   CT   TAT  WT
-----
1    0   10   10   10   0
2    0    5   15   15   0
3    0    8   23   23   0
4    0   12   35   35   0
5    0    6   41   41   0
=====
Rata-rata Turnaround Time: 24.00
Rata-rata Waiting Time : 16.00
Gantt Chart (FCFS):
| P1 | P2 | P3 | P4 | P5 |
0    10   15   23   35   41
=====
HASIL ALGORITMA SJF
=====
PID  AT   BT   CT   TAT  WT
-----
1    0   10   29   29   0
2    0    5    5    5    0
3    0    8   19   19   0
4    0   12   41   41   0
5    0    6   11   11   0
=====
Rata-rata Turnaround Time: 21.00
Rata-rata Waiting Time : 12.00
```

```
Gantt Chart (SJF):
| P1 | P2 | P3 | P4 | P5 |
0    29    5   19   41   11
=====
HASIL ALGORITMA Round Robin (Q=4)
=====
PID  AT   BT   CT   TAT  WT
-----
1    0   10   37   37   27
2    0    5   25   25   20
3    0    8   29   29   21
4    0   12   41   41   29
5    0    6   35   35   29
=====
Rata-rata Turnaround Time: 33.40
Rata-rata Waiting Time : 25.20
Gantt Chart (Round Robin (Q=4)):
| P1 | P2 | P3 | P4 | P5 |
0    37   25   29   41   35
=====
Hasil perbandingan disimpan ke file 'hasil_compare.txt'
=====
pradana@Bagus:~$ |
```


• Tes Case 3

```
pradana@Bagus:~$ nano input3.txt
pradana@Bagus:~$ cp input3.txt input.txt
./compare
=====
PROGRAM PERBANDINGAN ALGORITMA CPU SCHEDULING
=====
Membaca data dari file input.txt ...
=====
HASIL ALGORITMA FCFS
=====
PID  AT   BT   CT   TAT  WT
-----
1    0    8    8    8    0
2    2    4    12   10   6
3    4    9    21   17   8
4    6    5    26   20   15
5    8    3    29   21   18
-----
Rata-rata Turnaround Time: 15.20
Rata-rata Waiting Time   : 9.40

Gantt Chart (FCFS):
-----
| P1 | P2 | P3 | P4 | P5 |
-----
0    8    12   21   26   29
-----
HASIL ALGORITMA SJF
=====
PID  AT   BT   CT   TAT  WT
-----
1    0    8    8    8    0
2    2    4    15   13   9
3    4    9    29   25   16
4    6    5    20   14   9
5    8    3    11    3    0
-----
Rata-rata Turnaround Time: 12.60
Rata-rata Waiting Time   : 6.80
```

```
Gantt Chart (SJF):
-----
| P1 | P2 | P3 | P4 | P5 |
-----
0    8    15    29    20    11
-----
HASIL ALGORITMA Round Robin (Q=2)
=====
PID  AT   BT   CT   TAT  WT
-----
1    0    8    16   16   8
2    2    4    12   10   6
3    4    9    29   25   16
4    6    5    26   20   15
5    8    3    23   15   12
-----
Rata-rata Turnaround Time: 17.20
Rata-rata Waiting Time   : 11.40

Gantt Chart (Round Robin (Q=2)):
-----
| P1 | P2 | P3 | P4 | P5 |
-----
0    16    12    29    26    23
-----
Hasil perbandingan disimpan ke file 'hasil_compare.txt'
pradana@Bagus:~$ |
```

Test Case	Algoritma	Avg TAT	Avg WT
1	FCFS	...	...
1	SFJ	...	...
1	RR	...	...
2	FCFS	...	...
2	SFJ	...	...
2	RR	...	...
3	FCFS	...	...
3	SFJ	...	...
3	RR	...	...

Tugas 5 : Analisis Eksperimen perbandingan algoritma

1. Analisis Performa

- a. Dalam kondisi apa FCFS memberikan hasil terbaik?

Jawab:

FCFS paling baik atau cukup baik ketika proses datang *bersamaan* dan *burst time relatif seragam* tidak ada job yang sangat panjang yang menghalangi job lain

- b. Mengapa SJF hampir selalu memberikan Average WT terkecil?

Jawab:

SJF memprioritaskan proses dengan burst time terpendek → mengurangi jumlah proses yang menunggu lebih lama sebelum dieksekusi. Ini meminimalkan total waktu menunggu agregat dan dengan demikian rata-rata WT.

- c. Apa trade-off dari Round Robin?

Jawab:

tiap proses mendapat slice CPU secara bergiliran; bagus untuk sistem interaktif (respons cepat untuk banyak job kecil)

2. Context Switching

- a. Algoritma mana yang paling banyak melakukan context switch?

Jawab:

RR melakukan context switch paling banyak karena setiap time slice berakhir dan proses belum selesai

- b. Bagaimana pengaruh quantum terhadap jumlah context switch di Round Robin?

Jawab:

Semakin kecil quantum semakin banyak context switch.

Semakin besar quantum semakin sedikit context switch (mendekati FCFS ketika quantum $\geq$ burst terbesar).

- c. Coba jalankan RR dengan quantum 2, 4, 8, 16 — apa yang terjadi? (observasi umum)

- **Q = 2:** sangat responsif; job kecil cepat selesai; **banyak** context switches; WT untuk job panjang besar.
- **Q = 4:** compromise yang baik untuk banyak skenario interaktif; jumlah context switch menurun; respons masih baik.
- **Q = 8:** mendekati perilaku non-preemptive untuk job dengan $BT \leq 8$; context switch jauh lebih sedikit; job panjang mendapat slice besar → TAT/WT job panjang menurun, tapi job pendek yang tiba bisa sedikit lebih lama mendapat CPU.
- **Q = 16:** hampir seperti FCFS jika banyak job punya $BT < 16$; fairness berkurang, context switch sedikit.

3. Fairness

- a. Algoritma mana yang paling “adil” untuk semua proses?

Jawab:

Round Robin adalah yang paling adil karena setiap proses memperoleh jatah CPU secara berkala (time slice). SJF/FCFS tidak adil untuk job yang tiba terlambat atau yang panjang.

- b. Proses mana yang paling dirugikan di algoritma FCFS?

Jawab:

Proses yang datang terlambat tetapi memiliki burst kecil sering dirugikan bila ada proses awal yang sangat panjang (convoy effect). Contoh Test Case 1: P1 (BT besar) di depan membuat P2, P3, P4 menunggu lama sehingga WT mereka tinggi. Jadi yang paling dirugikan: proses kecil yang datang setelah job panjang.

- c. Apakah ada kemungkinan starvation di SJF?

Jawab:

Ya terutama pada preemptive SJF/SRTF: jika job pendek terus muncul, job panjang bisa terus dipreempt sehingga tidak pernah selesai (starvation).

4. Real World Application

- a. Untuk web server yang melayani banyak request kecil, algoritma mana yang cocok?

Jawab:

Round Robin atau algoritma berprioritas (sebenarnya fair queuing / priority scheduling untuk I/O bound/interactive tasks). Round Robin memastikan request kecil cepat diproses dan sistem responsif. Juga event-driven concurrency (non-blocking I/O) lebih umum di server nyata.

- b. Untuk render video (task besar), algoritma mana yang cocok?

Jawab:

FCFS atau SJF (batch scheduling) — job besar lebih baik dijalankan tanpa banyak preemption supaya efisiensi I/O/CPU maksimal. Dalam cluster/penjadwalan batch, SJF (atau heuristik berdasarkan job length) dapat meminimalkan average wait. Di dunia nyata, job scheduling di cluster biasanya memakai backfilling, priority, atau queue khusus job panjang.

- c. Untuk OS desktop (interactive), algoritma mana yang cocok?

Jawab:

Round Robin dengan quantum moderat (atau multilevel feedback queue — MLFQ) — agar interaktivitas tetap responsif sementara job latar belakang tetap mendapat CPU. MLFQ menyesuaikan quantum/priority berdasarkan behavior proses (IO-bound vs CPU-bound).

5. Rekomendasi & Praktikalitas (singkat untuk laporan)

Jawab:

Berdasarkan hasil analisis dan pengujian terhadap berbagai algoritma penjadwalan CPU, dapat disimpulkan bahwa setiap algoritma memiliki keunggulan dan kelemahan masing-masing tergantung pada tujuan dan karakteristik beban kerja. Jika tujuan utama sistem adalah meminimalkan *average waiting time*, maka algoritma **Shortest Job First (SJF)** atau **Shortest Remaining Time First (SRTF)** menjadi pilihan terbaik karena mampu menurunkan waktu tunggu rata-rata secara signifikan. Namun, algoritma ini memiliki potensi terjadinya *starvation* pada proses dengan durasi panjang sehingga dalam implementasi nyata perlu ditambahkan mekanisme **aging** untuk mencegah proses tertentu menunggu terlalu lama.

Apabila fokus sistem adalah pada **responsivitas dan keadilan pembagian waktu CPU**, maka algoritma **Round Robin (RR)** menjadi pilihan yang lebih tepat. Dengan pemilihan nilai *time quantum* yang sesuai (biasanya sedikit lebih besar dari rata-rata *CPU burst*), sistem dapat tetap responsif terhadap proses interaktif tanpa menimbulkan overhead *context switching* yang berlebihan. Sementara itu, untuk sistem **batch processing** atau tugas-tugas dengan durasi besar yang dijalankan secara berurutan seperti pada proses render video, algoritma **First Come First Served (FCFS)** atau SJF dapat memberikan efisiensi tinggi karena minim preemption dan overhead penjadwalan.